One-page architecture

Scope. Python 3.12 / FastAPI / Pydantic v2 / SQLAlchemy 2.x async + asyncpg / Alembic; PostgreSQL; Next.js App Router / TypeScript; SSE; pytest/httpx, Playwright, Compose. Applicant choices; Soup allows any stack. No broker/shared private cache. API contract defines bounds/errors; review records assumptions and unproven claims.

Identity. UUIDv4 projects/runs; one active high-entropy bearer key/project, SHA-256 hash stored, plaintext returned once. Project creation is local unauthenticated bootstrap, not account management. POST /runs/{id}/metrics accepts {batch_id,events[]} containing {event_id,metric,step,value,occurred_at}. Canonical normalized fields define identity within a run; batch array order matters because it orders new sequences.

Commit/retry. Each writer owns an AsyncSession and explicit READ COMMITTED transaction including auth. Lock authenticated project FOR SHARE, revalidate key, then scoped run FOR UPDATE; rotation takes project FOR UPDATE. Compare identities, insert new events, allocate contiguous seq from runs.last_seq, and persist receipt/counter atomically. Exact batch retry returns original receipt; changed batch/conflicting event returns 409 without writes. Identical cross-batch events reuse seq. Rollback restores counter; no SERIAL/process lock. Return 2xx after commit; uncertain acknowledgement requires identical retry.

Snapshot/UI. Short read-only REPEATABLE READ returns run metadata, high-water H, event count and current values together. Maximum 128 metric names/run bounds this summary; raw history remains in DB. Greatest (step,seq) wins, never timestamp/arrival. Install summary/H atomically; chart bootstrap replays from 0 into a 2,000-event window while retaining snapshot winners/count. Reconnect after last applied stream cursor, preserving chart history; gap commits replay. Apply contiguous events once, including late events changing only count/cursor. Ignore duplicates; gaps/malformed frames never advance cursor.

Replay/resources. Authenticate GET /runs/{id}/events before headers; Last-Event-ID is last applied seq (absent=0). Read scoped committed seq > cursor ascending, pages <=250, release sessions before sends. Events/receipts/counter survive API restart; no pruning. No client queue; one bounded page, 5s ASGI-send deadline and independent 1s key watchdog/1s query deadline, failing closed. Target rotation stops new sends within 2s under responsive scheduling; buffered bytes cannot be recalled. Cancellation releases resources; bounds need measurement.

Browser/isolation. FastAPI owns all APIs/SSE; Next.js supplies client pages, no second backend. Direct public-origin fetch uses bearer header, no-store, exact CORS; key only in ephemeral memory. No keys in public env, URLs, cookies, storage, HTML or logs. Fetch/ReadableStream parses bounded complete frames; advance reconnect cursor only with applied state. Abort on key/run change; backoff transient failures, stop on 401/403/auth-expired. Show stale/reconnecting until caught-up marker. Every lookup derives scope from key, including rotation/replay/list; foreign/missing runs share 404, all responses no-store.

Proof/limits. Executed PostgreSQL/HTTP/browser checks cover retries, rollback, UI/DB equality, gap/restart replay, A/B state, slow reader and rotation. The local50x10/s benchmark measured498.92/498.17 events/s, not exact500; scoped SHIP uses a self-defined1% tolerance. Audit and evidence distinguish verified cases from remaining security/capacity limits. Architecture intent alone is not proof.